



Legacy Error Processing

Version 2026.1
2026-04-20

Legacy Error Processing

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

Legacy Error Processing	1
1 How Traditional Error Processing Works	1
1.1 Internal Error-Trapping Behavior	1
1.2 Current Context Level	2
1.3 Error Codes	3
2 Handling Errors with \$ZTRAP	5
2.1 Setting \$ZTRAP in a Procedure	5
2.2 Setting \$ZTRAP in a Routine	5
2.3 Writing \$ZTRAP Code	6
2.4 Using \$ZTRAP	6
2.5 Unstacking NEW Commands With Error Traps	6
2.6 \$ZTRAP Flow of Control Options	7
3 Handling Errors with \$ETRAP	8
3.1 \$ETRAP Error Handlers	9
3.2 Context-specific \$ETRAP Error Handlers	9
3.3 \$ETRAP Flow of Control Options	10
4 Handling Errors in an Error Handler	11
4.1 Errors in a \$ZTRAP Error Handler	11
4.2 Errors in a \$ETRAP Error Handler	12
4.3 Error Information in the \$ZERROR and \$ECODE Special Variables	12
5 Forcing an Error	12
5.1 Setting \$ECODE	12
5.2 Creating Application-Specific Errors	13
6 Processing Errors at the Terminal Prompt	13
6.1 Understanding Error Message Formats	13
6.2 Understanding the Terminal Prompt	14
6.3 Recovering from the Error	14
 List of Figures	
Figure 1: Frames on a Call Stack	2
Figure 2: \$ZTRAP Error Handlers	8
Figure 3: \$ETRAP Error Handlers	10

Legacy Error Processing

This page describes error processing that uses `$ZTRAP`, a form of error processing that may be encountered in legacy applications. New applications should use [TRY-CATCH](#) instead.

1 How Traditional Error Processing Works

For traditional error processing, InterSystems IRIS® data platform enables your application to have an *error handler*. An error handler processes any error that may occur while the application is running. A special variable specifies the ObjectScript commands to be executed when an error occurs. These commands may handle the error directly or may call a routine to handle it.

To set up an error handler, the basic process is:

1. Create one or more routines to perform error processing. Write code to perform error processing. This can be general code for the entire application or specific processing for specific error conditions. This allows you to perform customized error handling for each particular part of an application.
2. Establish one or more error handlers within your application, each using specific appropriate error processing.

If an error occurs and no error handler has been established, the behavior depends on how the InterSystems IRIS session was started:

1. If you signed onto InterSystems IRIS at the Terminal prompt and have not set an error trap, InterSystems IRIS displays an error message on the principal device and returns the Terminal prompt with the program stack intact. The programmer can later resume execution of the program.
2. If you invoked InterSystems IRIS in Application Mode and have not set an error trap, InterSystems IRIS displays an error message on the principal device and executes a **HALT** command.

1.1 Internal Error-Trapping Behavior

To get the full benefit of InterSystems IRIS error processing and the scoping issues surrounding the **\$ZTRAP** special variable (as well as **\$ETRAP**), it is helpful to understand how InterSystems IRIS transfers control from one routine to another.

InterSystems IRIS builds a data structure called a “context frame” each time any of the following occurs:

- A routine calls another routine with a **DO** command. (This kind of frame is also known as a “**DO** frame.”)
- An **XECUTE** command argument causes ObjectScript code to execute. (This kind of frame is also known as a “**XECUTE** frame.”)
- A user-defined function is executed.

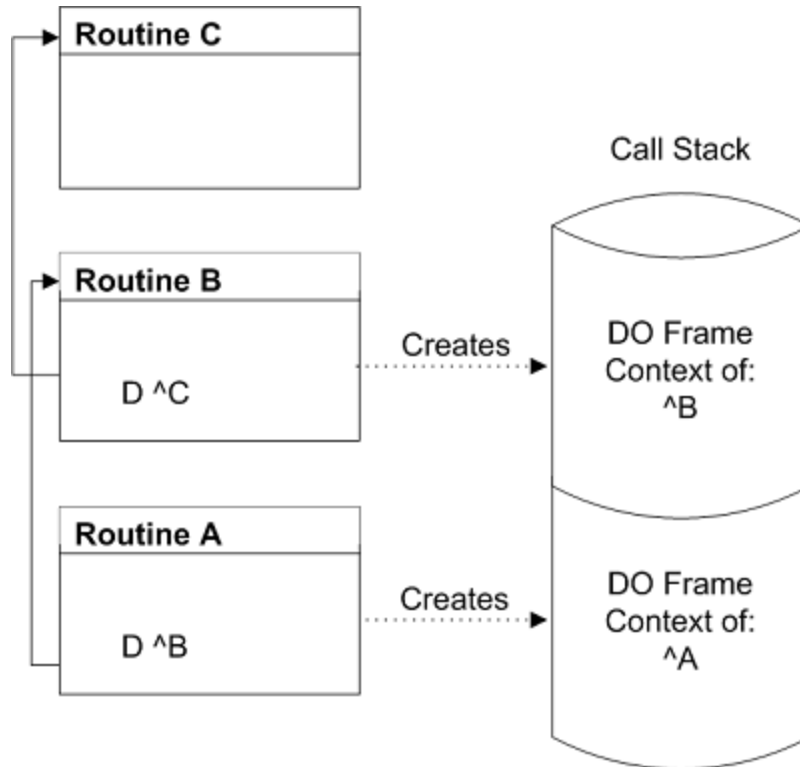
The frame is built on the call stack, one of the private data structures in the address space of your process. InterSystems IRIS stores the following elements in the frame for a routine:

- The value of the **\$ZTRAP** special variable (if any)
- The value of the **\$ETRAP** special variable (if any)

- The position to return from the subroutine

When routine A calls routine B with `DO ^B`, InterSystems IRIS builds a **DO** frame on the call stack to preserve the context of A. When routine B calls routine C, InterSystems IRIS adds a **DO** frame to the call stack to preserve the context of B, and so forth.

Figure 1: Frames on a Call Stack



If routine A in the figure above is invoked at the Terminal prompt using the **DO** command, then an extra **DO** frame, not described in the figure, exists at the base of the call stack.

1.2 Current Context Level

You can use the following to return information about the current context level:

- The **\$STACK** special variable contains the current relative stack level.
- The **\$ESTACK** special variable contains the current stack level. It can be initialized to 0 (level zero) at any user-specified point.
- The **\$STACK** function returns information about the current context and contexts that have been saved on the call stack

1.2.1 The \$STACK Special Variable

The **\$STACK** special variable contains the number of frames currently saved on the call stack for your process. The **\$STACK** value is essentially the context level number (zero based) of the currently executing context. Therefore, when an image is started, but before any commands are processed, the value of **\$STACK** is 0.

See the [\\$STACK](#) special variable for details.

1.2.2 The \$ESTACK Special Variable

The **\$ESTACK** special variable is similar to the **\$STACK** special variable, but is more useful in error handling because you can reset it to 0 (and save its previous value) with the **NEW** command. Thus, a process can reset **\$ESTACK** in a particular context to mark it as a **\$ESTACK** level 0 context. Later, if an error occurs, error handlers can test the value of **\$ESTACK** to unwind the call stack back to that context.

See the [\\$ESTACK](#) special variable for details.

1.2.3 The \$STACK Function

The **\$STACK** function returns information about the current context and contexts that have been saved on the call stack. For each context, the **\$STACK** function provides the following information:

- The type of context (**DO**, **XECUTE**, or user-defined function)
- The entry reference and command number of the last command processed in the context
- The source routine line or **XECUTE** string that contains the last command processed in the context
- The **\$ECODE** value of any error that occurred in the context (available only during error processing when **\$ECODE** is non-null)

When an error occurs, all context information is immediately saved on your process error stack. The context information is then accessible by the **\$STACK** function until the value of **\$ECODE** is cleared by an error handler. In other words, while the value of **\$ECODE** is non-null, the **\$STACK** function returns information about a context saved on the error stack rather than an active context at the same specified context level.

See the [\\$STACK](#) function for details.

When an error occurs and an error stack already exists, InterSystems IRIS records information about the new error at the context level where the error occurred, unless information about another error already exists at that context level on the error stack. In this case, the information is placed at the next level on the error stack (regardless of the information that may already be recorded there).

Therefore, depending on the context level of the new error, the error stack may extend (one or more context levels added) or information at an existing error stack context level may be overwritten to accommodate information about the new error.

Keep in mind that you clear your process error stack by clearing the **\$ECODE** special variable.

1.3 Error Codes

When an error occurs, InterSystems IRIS sets the **\$ZERROR** and **\$ECODE** special variables to a value describing the error. The **\$ZERROR** and **\$ECODE** values are intended for use immediately following an error. Because these values may not be preserved across routine calls, users who wish to preserve a value for later use should copy it to a variable.

Important: Do not use error code values to try to detect error conditions. To *detect* error conditions, use a [TRY-CATCH block](#) or [\\$ZTRAP](#), which are designed for that purpose.

1.3.1 \$ZERROR Value

InterSystems IRIS sets **\$ZERROR** to a string containing:

- The InterSystems IRIS error code, enclosed in angle brackets.
- The [label](#), offset, and routine name where the error occurred.
- (For some errors): Additional information, such as the name of the item that caused the error.

The **AsSystemError()** method of the %Exception.SystemException class returns the same values in the same format as **\$ZERROR**.

The following examples show the type of messages to which **\$ZERROR** is set when InterSystems IRIS encounters an error. In the following example, the undefined local variable *abc* is invoked at line offset 2 from label PrintResult of routine MyTest. **\$ZERROR** contains:

```
<UNDEFINED>PrintResult+2^MyTest *abc
```

The following error occurred when a non-existent class is invoked at line offset 3:

```
<CLASS DOES NOT EXIST>PrintResult+3^MyTest *%SYSTEM.XXQL
```

The following error occurred when a non-existent method of an existing class is invoked at line offset 4:

```
<METHOD DOES NOT EXIST>PrintResult+4^MyTest *BadMethod,%SYSTEM.SQL
```

You can also explicitly set the special variable **\$ZERROR** as any string up to 128 characters; for example:

ObjectScript

```
SET $ZERROR="Any String"
```

The **\$ZERROR** value is intended for use immediately following an error. Because a **\$ZERROR** value may not be preserved across routine calls, users that wish to preserve a **\$ZERROR** value for later use should copy it to a variable. It is strongly recommended that users set **\$ZERROR** to the null string ("") immediately after use. See the [\\$ZERROR](#) special variable for details. For further information on handling **\$ZERROR** errors, refer to the %SYSTEM.Error class methods in the *InterSystems Class Reference*.

1.3.2 \$ECODE Value

When an error occurs, InterSystems IRIS sets **\$ECODE** to the value of a comma-surrounded string containing the ANSI Standard error code that corresponds to the error. For example, when you make a reference to an undefined global variable, InterSystems IRIS sets **\$ECODE** set to the following string:

```
,M7,
```

If the error has no corresponding ANSI Standard error code, InterSystems IRIS sets **\$ECODE** to the value of a comma-surrounded string containing the InterSystems IRIS error code preceded by the letter Z. For example, if a process has exhausted its symbol table space, InterSystems IRIS places the error code <STORE> in the **\$ZERROR** special variable and sets **\$ECODE** to this string:

```
,ZSTORE,
```

After an error occurs, your error handlers can test for specific error codes by examining the value of the **\$ZERROR** special variable or the **\$ECODE** special variable.

Note: Error handlers should examine **\$ZERROR** rather than **\$ECODE** special variable for specific errors.

See the [\\$ECODE](#) special variable for details.

2 Handling Errors with \$ZTRAP

To handle errors with **\$ZTRAP**, you set the **\$ZTRAP** special variable to a *location*, specified as a quoted string. You set the **\$ZTRAP** special variable to an entry reference that specifies the *location* to which control is to be transferred when an error occurs. You then write **\$ZTRAP** code at that location.

When you set **\$ZTRAP** to a non-empty value, it takes precedence over any existing **\$ETRAP** error handler. InterSystems IRIS implicitly performs a **NEW \$ETRAP** command and sets **\$ETRAP** equal to "".

2.1 Setting \$ZTRAP in a Procedure

Within a procedure, you can only set the **\$ZTRAP** special variable to a line label (private label) within that procedure. You cannot set **\$ZTRAP** to any external routine from within a procedure block.

When displaying the **\$ZTRAP** value, InterSystems IRIS does not return the name of the private label. Instead, it returns the offset from the top of the procedure where that private label is located.

For further details see the [\\$ZTRAP](#) special variable.

2.2 Setting \$ZTRAP in a Routine

Within a routine, you can set the **\$ZTRAP** special variable to a label in the current routine, to an external routine, or to a label within an external routine. You can only reference an external routine if the routine is not procedure block code. The following example establishes LogErr^ErrRou as the error handler. When an error occurs, InterSystems IRIS executes the code found at the LogErr label within the ^ErrRou routine:

ObjectScript

```
SET $ZTRAP="LogErr^ErrRou"
```

When displaying the **\$ZTRAP** value, InterSystems IRIS displays the label name and (when appropriate) the routine name.

A label name must be unique within its first 31 characters. Label names and routine names are case-sensitive.

Within a routine, **\$ZTRAP** has three forms:

- **SET \$ZTRAP="location"**
- **SET \$ZTRAP="*location"** which executes in the context in which the error occurred that invoked it.
- **SET \$ZTRAP="^%ETN"** which executes the system-supplied error routine **^%ETN** in the context in which the error occurred that invoked it. You cannot execute **^%ETN** (or any external routine) from a procedure block. Either specify the code is [\[Not ProcedureBlock\]](#), or use a routine such as the following, which invokes the **^%ETN** entry point **BACK^%ETN**:

```
ClassMethod MyTest() as %Status
{
    SET $ZTRAP="Error"
    SET ans = 5/0 /* divide-by-zero error */
    WRITE "Exiting ##class(User.A).MyTest()",!
    QUIT ans
Error
    SET err=$ZERROR
    SET $ZTRAP=""
    DO BACK^%ETN
    QUIT $$$ERROR($$CacheError,err)
}
```

For more information on **^%ETN** and its entry points, see [\(Legacy\) Using ^%ETN for Error Logging](#). For details on its use with **\$ZTRAP**, see [SET \\$ZTRAP=^%ETN](#).

For further details see the [\\$ZTRAP](#) special variable.

2.3 Writing \$ZTRAP Code

The *location* that **\$ZTRAP** points to can perform a variety of operations to display, log, and/or correct an error. Regardless of what error handling operations you wish to perform, the **\$ZTRAP** code should begin by performing two tasks:

- Set **\$ZTRAP** to another value, either the *location* of an error handler, or the empty string (""). (You must use **SET**, because you cannot **KILL \$ZTRAP**.) This is done because if another error occurs during error handling, that error would invoke the current **\$ZTRAP** error handler. If the current error handler is the error handler you are in, this would result in an infinite loop.
- Set a variable to **\$ZERROR**. If you wish to reference a **\$ZERROR** value later in your code, refer to this variable, not **\$ZERROR** itself. This is done because **\$ZERROR** contains the most-recent error, and a **\$ZERROR** value may not be preserved across routine calls, including internal routine calls. If another error occurs during error handling, the **\$ZERROR** value would be overwritten by that new error.

It is strongly recommended that users set **\$ZERROR** to the null string ("") immediately after use.

The following example shows these essential **\$ZTRAP** code statements:

ObjectScript

```
MyErrorHandler
SET $ZTRAP=""
SET err=$ZERROR
/* error handling code
   using err as the error
   to be handled */
```

2.4 Using \$ZTRAP

Each routine in an application can establish its own **\$ZTRAP** error handler by setting **\$ZTRAP**. When an error trap occurs, InterSystems IRIS takes the following steps:

1. Sets the special variable **\$ZERROR** to an error message.
2. Resets the program stack to the state it was in when the error trap was set (when the **SET \$ZTRAP=** was executed). In other words, the system removes all entries on the stack until it reaches the point at which the error trap was set. (The program stack is not reset if **\$ZTRAP** was set to a string beginning with an asterisk (*).)
3. Resumes the program at the location specified by the value of **\$ZTRAP**. The value of **\$ZTRAP** remains the same.

Note: You can explicitly set the variable **\$ZERROR** as any string up to 128 characters. Usually you would set **\$ZERROR** to a null string, but you can set **\$ZERROR** to a value.

2.5 Unstacking NEW Commands With Error Traps

When an error trap occurs and the program stack entries are removed, InterSystems IRIS also removes all stacked **NEW** commands back to the subroutine level containing the **SET \$ZTRAP=**. However, all **NEW** commands executed at that subroutine level remain, regardless of whether they were added to the stack before or after **\$ZTRAP** was set.

For example:

ObjectScript

```

Main
    SET A=1, B=2, C=3, D=4, E=5, F=6
    NEW A, B
    SET $ZTRAP="ErrSub"
    NEW C, D
    DO Sub1
    RETURN
Sub1()
    NEW E, F
    WRITE 6/0    // Error: division by zero
    RETURN
ErrSub()
    WRITE !, "Error is: ", $ZERROR
    WRITE
    RETURN

```

When the error in Sub1 activates the error trap, the former values of E and F stacked in Sub1 are removed, but A, B, C, and D remain stacked.

2.6 \$ZTRAP Flow of Control Options

After a **\$ZTRAP** error handler has been invoked to handle an error and has performed any cleanup or error logging operations, the error handler has three flow control options:

- Handle the error and continue the application.
- Pass control to another error handler
- Terminate the application

2.6.1 Continuing the Application

After a **\$ZTRAP** error handler has handled an error, you can continue the application by issuing a **GOTO**. You do not have to clear the values of the **\$ZERROR** or **\$ECODE** special variables to continue normal application processing. However, you should clear **\$ZTRAP** (by setting it to the empty string) to avoid a possible infinite error handling loop if another error occurs. See “[Handling Errors in an Error Handler](#)” for more information.

After completing error processing, your **\$ZTRAP** error handler can use the **GOTO** command to transfer control to a pre-determined restart or continuation point in your application to resume normal application processing.

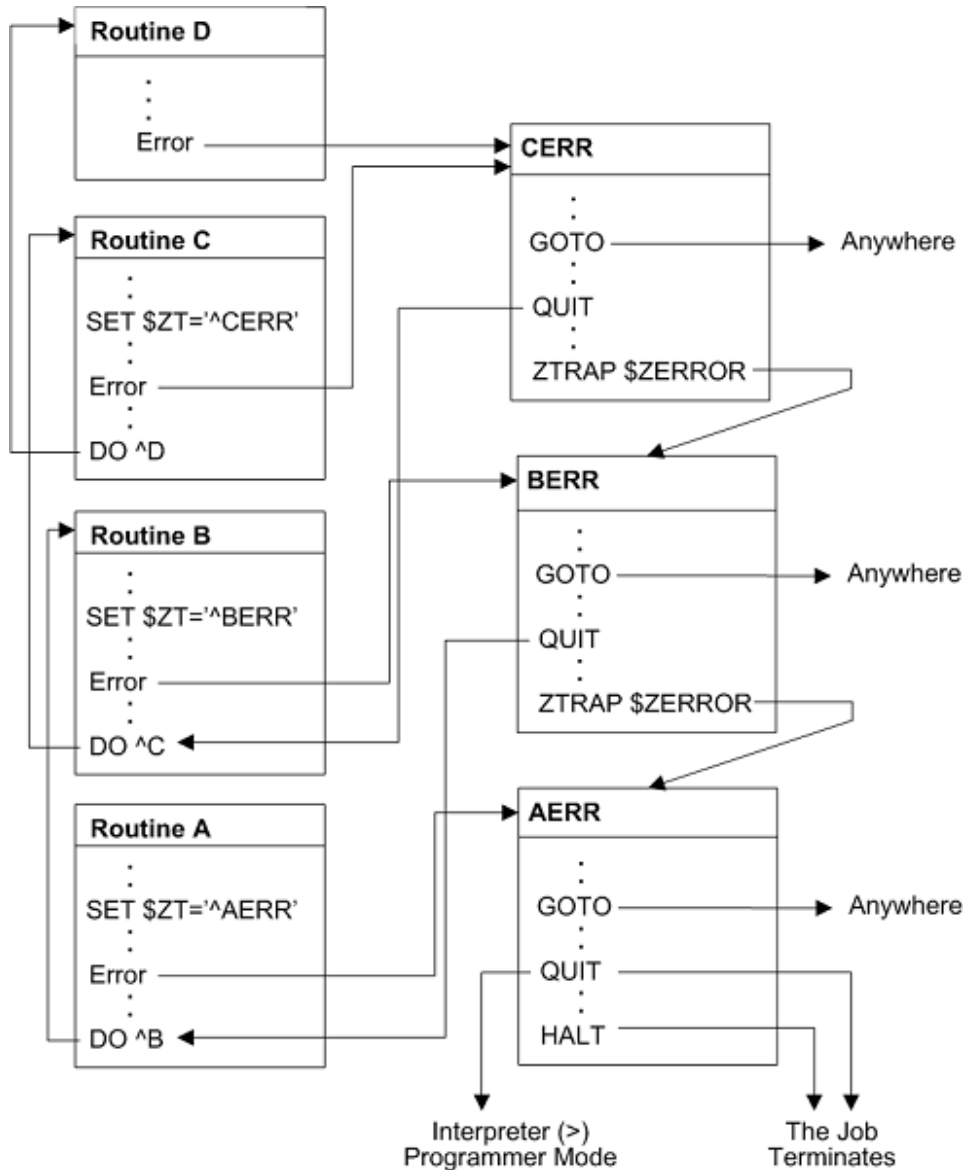
When an error handler has handled an error, the **\$ZERROR** special variable is set to a value. This value is not necessarily cleared when the error handler completes. Some routines reset **\$ZERROR** to the null string. The **\$ZERROR** value is overwritten when the next error occurs that invokes an error handler. For this reason, the **\$ZERROR** value should only be accessed within the context of an error handler. If you wish to preserve this value, copy it to a variable and reference that variable, not **\$ZERROR** itself. Accessing **\$ZERROR** in any other context does not produce reliable results.

2.6.2 Passing Control to Another Error Handler

If the error condition cannot be corrected by a **\$ZTRAP** error handler, you can use a special form of the **ZTRAP** command to transfer control to another error handler. The command **ZTRAP \$ZERROR** re-signals the error condition and causes InterSystems IRIS to unwind the call stack to the next call stack level with an error handler. After InterSystems IRIS has unwound the call stack to the level of the next error handler, processing continues in that error handler. The next error handler may have been set by either a **\$ZTRAP** or a **\$ETRAP**.

The following figure shows the flow of control in **\$ZTRAP** error handling routines.

Figure 2: \$ZTRAP Error Handlers



3 Handling Errors with \$ETRAP

When an error trap occurs and you have set **\$ETRAP**, InterSystems IRIS takes the following steps:

1. Sets the values of **\$ECODE** and **\$ZERROR**.
2. Processes the commands that are the value of **\$ETRAP**.

By default, each **DO**, **XECUTE**, or user-defined function context inherits the **\$ETRAP** error handler of the frame that invoked it. This means that the designated **\$ETRAP** error handler at any context level is the last defined **\$ETRAP**, even if that definition was made several stack levels down from the current level.

3.1 \$ETRAP Error Handlers

The **\$ETRAP** special variable can contain one or more ObjectScript commands that are executed when an error occurs. Use the **SET** command to set **\$ETRAP** to a string that contains one or more InterSystems IRIS commands that transfer control to an error-handling routine. This example transfers control to the LogError code label (which is part of the routine ErrRoutine):

ObjectScript

```
SET $ETRAP="DO LogError^ErrRoutine"
```

The commands in the **\$ETRAP** special variable are always followed by an implicit **QUIT** command. The implicit **QUIT** command quits with a null string argument when the **\$ETRAP** error handler is invoked in a user-defined function context where a **QUIT** with arguments is required.

\$ETRAP has a global scope. This means that setting **\$ETRAP** should usually be preceded by **NEW \$ETRAP**. Otherwise, if the value of **\$ETRAP** is set in the current context, then, after passing beyond the scope of that context, the value stored in **\$ETRAP** is still present while control is in a higher-level context. Thus, if you do not specify the **NEW \$ETRAP**, then **\$ETRAP** could be executed at an unexpected time when the context that set that it no longer exists.

See the [\\$ETRAP](#) special variable for details.

3.2 Context-specific \$ETRAP Error Handlers

Any context can establish its own **\$ETRAP** error handler by taking the following steps:

1. Use the **NEW** command to create a new copy of **\$ETRAP**.
2. Set **\$ETRAP** to a new value.

If a routine sets **\$ETRAP** without first creating a new copy of **\$ETRAP**, a new **\$ETRAP** error handler is established for the current context, the context that invoked it, and possibly other contexts that have been saved on the call stack. Therefore InterSystems recommends that you create a new copy of **\$ETRAP** before you set it.

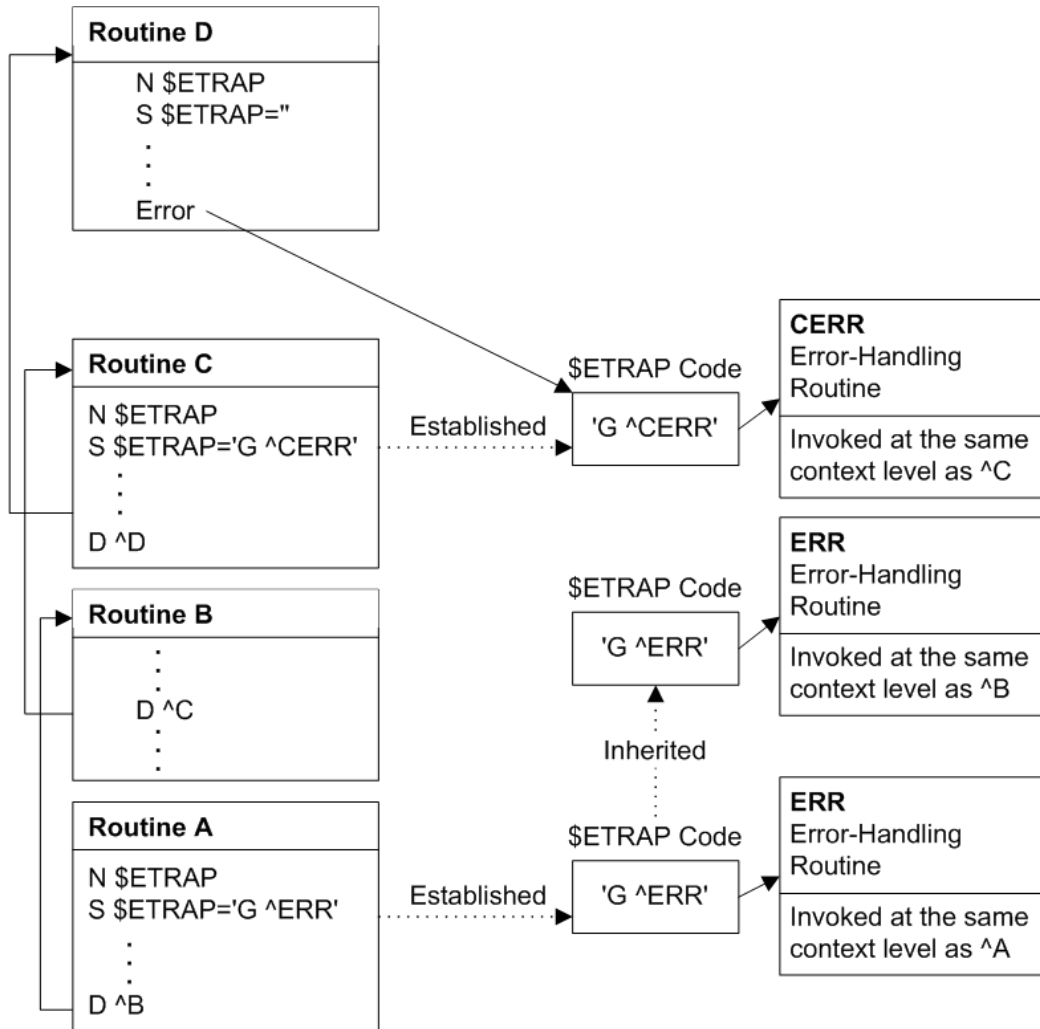
Keep in mind that creating a new copy of **\$ETRAP** does not clear **\$ETRAP**. The value of **\$ETRAP** remains unchanged by the **NEW** command.

The figure below shows the sequence of **\$ETRAP** assignments that create the stack of **\$ETRAP** error handlers. As the figure shows:

- Routine A creates a new copy of **\$ETRAP**, sets it to “GOTO ^ERR”, and contains the **DO** command to call routine B.
- Routine B does nothing with **\$ETRAP** (thereby inheriting the **\$ETRAP** error handler of Routine A) and contains the **DO** command to call routine C.
- Routine C creates a new copy of **\$ETRAP**, sets it to “GOTO ^CERR”, and contains the **DO** command to call routine D.
- Routine D creates a new copy of **\$ETRAP** and then clears it, leaving no **\$ETRAP** error handler for its context.

If an error occurs in routine D (a context in which no **\$ETRAP** error handler is defined), InterSystems IRIS removes the **DO** frame for routine D from the call stack and transfers control to the **\$ETRAP** error handler of Routine C. The **\$ETRAP** error handler of Routine C, in turn, dispatches to ^CERR to process the error. If an error occurs in Routine C, InterSystems IRIS transfers control to the **\$ETRAP** error handler of Routine C, but does not unwind the stack because the error occurs in a context where a **\$ETRAP** error handler is defined.

Figure 3: \$ETRAP Error Handlers



3.3 \$ETRAP Flow of Control Options

When the **\$ETRAP** error handler has been invoked to handle an error and perform any cleanup or error-logging operations, it has the following flow-of-control options:

- Handle the error and continue the application.
- Pass control to another error handler.
- Terminate the application.

3.3.1 Handling the Error and Continuing the Application

When a **\$ETRAP** error handler is called to handle an error, InterSystems IRIS considers the error condition active until the error condition is dismissed. You dismiss the error condition by setting the **\$ECODE** special variable to the null string:

ObjectScript

```
SET $ECODE=""
```

Clearing **\$ECODE** also clears the error stack for your process.

Typically, you use the **GOTO** command to transfer control to a predetermined restart or continuation point in your application after the error condition is dismissed. In some cases, you may find it more convenient to quit back to the previous context level after dismissing the error condition.

3.3.2 Passing Control to Another Error Handler

If the error condition is not dismissed, InterSystems IRIS passes control to another error handler on the call stack when a **QUIT** command terminates the context at which the **\$ETRAP** error handler was invoked. Therefore, you pass control to a previous level error handler by performing a **QUIT** from a **\$ETRAP** context without clearing **\$ECODE**.

If routine D, called from routine C, contains an error that transfers control to **^CERR**, the **QUIT** command in **^CERR** that is not preceded by setting **\$ECODE** to "" (the empty string) transfers control to the **\$ETRAP** error handler at the previous context level. In contrast, if the error condition is dismissed by clearing **\$ECODE**, a **QUIT** from **^CERR** transfers control to the statement in routine B that follows the **DO ^C** command.

3.3.3 Terminating the Application

If no previous level error handlers exist on the call stack and a **\$ETRAP** error handler performs a **QUIT** without dismissing the error condition, the application is terminated. In Application Mode, InterSystems IRIS is then run down and control is passed to the operating system. The Terminal prompt then appears.

Keep in mind that you use the **QUIT** command to terminate a **\$ETRAP** error handler context whether or not the error condition is dismissed. Because the same **\$ETRAP** error handler can be invoked at context levels that require an argumentless **QUIT** and at context levels (user-defined function contexts) that require a **QUIT** with arguments, the **\$QUIT** special variable is provided to indicate the **QUIT** command form required at a particular context level.

The **\$QUIT** special variable returns 1 (one) for contexts that require a **QUIT** with arguments and returns 0 (zero) for contexts that require an argumentless **QUIT**.

A **\$ETRAP** error handler can use **\$QUIT** to provide for either circumstance as follows:

ObjectScript

```
Quit:$QUIT "" Quit
```

When appropriate, a **\$ETRAP** error handler can terminate the application using the **HALT** command.

4 Handling Errors in an Error Handler

When an error occurs in an error handler, the flow of execution depends on the type of error handler that is currently executing.

4.1 Errors in a \$ZTRAP Error Handler

If the new error occurs in a **\$ZTRAP** error handler, InterSystems IRIS passes control to the first error handler it encounters, unwinding the call stack only if necessary. Therefore, if the **\$ZTRAP** error does not clear **\$ZTRAP** at the current stack level and another error subsequently occurs in the error handler, the **\$ZTRAP** handler is invoked again at the same context level, causing an infinite loop. To avoid this, Set **\$ZTRAP** to another value at the beginning of the error handler.

4.2 Errors in a \$ETRAP Error Handler

If the new error occurs in a **\$ETRAP** error handler, InterSystems IRIS unwinds the call stack until the context level at which the **\$ETRAP** error handler was invoked has been removed. InterSystems IRIS then passes control to the next error handler (if any) on the call stack.

4.3 Error Information in the \$ZERROR and \$ECODE Special Variables

If another error occurs during the handling of the original error, information about the second error replaces the information about the original error in the **\$ZERROR** special variable. However, InterSystems IRIS appends the new information to the **\$ECODE** special variable. Depending on the context level of the second error, InterSystems IRIS may append the new information to the process error stack as well.

If the existing value of the **\$ECODE** special variable is non-null, InterSystems IRIS appends the code for the new error to the current **\$ECODE** value as a new comma piece. Error codes accrue in the **\$ECODE** special variable until either of the following occurs:

- You explicitly clear **\$ECODE**, for example:

ObjectScript

```
SET $ECODE = ""
```

- The length of **\$ECODE** exceeds the maximum string length.

Then, the next new error code replaces the current list of error codes in **\$ECODE**.

When an error occurs and an error stack already exists, InterSystems IRIS records information about the new error at the context level where the error occurred, unless information about another error already exists at that context level on the error stack. In this case, the information is placed at the next level on the error stack (regardless of the information that may already be recorded there).

Therefore, depending on the context level of the new error, the error stack may extend (one or more context levels added) or information at an existing error stack context level may be overwritten to accommodate information about the new error.

Keep in mind that you clear your process error stack by clearing the **\$ECODE** special variable.

See the [\\$ECODE](#) and [\\$ZERROR](#) special variables for details. For further information on handling **\$ZERROR** errors, refer to the %SYSTEM.Error class methods in the *InterSystems Class Reference*.

5 Forcing an Error

You set the **\$ECODE** special variable or use the **ZTRAP** command to cause an error to occur under controlled circumstances.

5.1 Setting \$ECODE

You can set the **\$ECODE** special variable to any non-null string to cause an error to occur. When your routine sets **\$ECODE** to a non-null string, InterSystems IRIS sets **\$ECODE** to the specified string and then generates an error condition. The **\$ZERROR** special variable in this circumstance is set with the following error text:

```
<ECODETRAP>
```

Control then passes to error handlers as it does for normal application-level errors.

You can add logic to your error handlers to check for errors caused by setting **\$ECODE**. Your error handler can check **\$ZERROR** for an <ECODETRAP> error (for example, “\$ZE[“ECODETRAP”]”) or your error handler can check **\$ECODE** for a particular string value that you choose.

5.2 Creating Application-Specific Errors

Keep in mind that the ANSI Standard format for **\$ECODE** is a comma-surrounded list of one or more error codes:

- Errors prefixed with “Z” are implementation-specific errors
- Errors prefixed with “U” are application-specific errors

You can create your own error codes following the ANSI Standard by having the error handler set **\$ECODE** to the appropriate error message prefixed with a “U”.

ObjectScript

```
SET $ECODE=",Upassword expired,"
```

6 Processing Errors at the Terminal Prompt

When you generate an error after you sign onto InterSystems IRIS at the Terminal prompt with no error handler set, InterSystems IRIS takes the following steps when an error occurs in a line of code you enter:

1. InterSystems IRIS displays an error message on the process’s principal device.
2. The process breaks at the call stack level where the error occurred.
3. The process returns the Terminal prompt.

6.1 Understanding Error Message Formats

As an error message, InterSystems IRIS displays three lines:

1. The entire line of source code in which the error occurred.
2. Below the source code line, a caret (^) points to the command that caused the error.
3. A line containing the contents of **\$ZERROR**.

In the following Terminal prompt example, the second **SET** command has an undefined local variable error:

Terminal

```
USER>WRITE "hello",! SET x="world" SET y=zzz WRITE x,!
hello

WRITE "hello",! SET x="world" SET y=zzz WRITE x,!
                                ^
<UNDEFINED> *zzz
USER>
```

In the following example, the same line of code is in a program named **mytest** executed from the Terminal prompt:

Terminal

```
USER>DO ^mytest
hello

WRITE "hello",! SET x="world" SET y=zzz WRITE x,!
^
<UNDEFINED>WriteOut+2^mytest *zzz
USER 2d0>
```

In this case, **\$ZERROR** indicates that the error occurred in `mytest` at an offset of 2 lines from the a label named `WriteOut`. Note that the prompt has changed, indicating that a new program stack level has been initiated.

6.2 Understanding the Terminal Prompt

By default, the Terminal prompt specifies the current namespace. If one or more transactions are open, it also includes the **\$TLEVEL** transaction level count. This default prompt can be configured with different contents, as described in the **ZNSPACE** command documentation. The following examples show the defaults:

Terminal

```
USER>
```

Terminal

```
TL1:USER>
```

If an error occurs during the execution of a routine, the system saves the current program stack and initiates a new stack frame. An extended prompt appears, such as:

Terminal

```
USER 2d0>
```

This extended prompt indicates that there are two entries on the program stack, the last of which is an invoking of **DO** (as indicated by the “d”). Note that this error placed two entries on the program stack. The next **DO** execution error would result in the prompt:

Terminal

```
USER 4d0>
```

For a more detailed explanation, see [Terminal Prompt Shows Program Stack Information](#).

6.3 Recovering from the Error

You can then take any of the following steps:

- Issue commands from the Terminal prompt
- View and modify your variables and global data
- Edit the routine containing the error or any other routine
- Execute other routines

Any of these steps can even cause additional errors.

After you have taken these steps, your most likely course is to either resume execution or to delete all or part of the program stack.

6.3.1 Resuming Execution at the Next Sequential Command

You can resume execution at the next command after the command that caused the error by entering an argumentless **GOTO** from the Terminal prompt:

Terminal

```
USER>D0 ^mytest
hello

    WRITE "hello",! SET x="world" SET y=zzz WRITE x,!
                                ^
<UNDEFINED>WriteOut+2^mytest *zzz
USER 2d0>GOTO
world

USER>
```

6.3.2 Resuming Execution at Another Line

You can resume execution at another line by issuing a **GOTO** with a label argument from the Terminal prompt:

Terminal

```
USER 2d0>GOTO ErrSect
```

6.3.3 Deleting the Program Stack

You can delete the entire program stack by issuing an argumentless **QUIT** command from the Terminal prompt:

Terminal

```
USER 4d0>QUIT
USER>
```

6.3.4 Deleting Part of the Program Stack

You can issue **QUIT** *n* with an integer argument from the Terminal prompt to delete the last (or last several) program stack entry:

Terminal

```
USER 8d0>QUIT 1
USER 7E0>QUIT 3
USER 4d0>QUIT 1
USER 3E0>QUIT 1
USER 2d0>QUIT 1
USER 1S0>QUIT 1
USER>
```

Note that in this example because the program error created two program stack entries, you must be on a “d” stack entry to resume execution by issuing a **GOTO**. Depending on what else has occurred, a “d” stack entry may be even-numbered (USER 2d0>) or odd-numbered (USER 3d0>).

By using **NEW \$ESTACK**, you can quit to a specified program stack level:

Terminal

```
USER 4d0>NEW $ESTACK
```

```
USER 5E1>  
/* more errors create more stack frames */
```

```
USER 11d7>QUIT $ESTACK
```

```
USER 4d0>
```

Note that the **NEW \$ESTACK** command adds one entry to the program stack.